# SIDARTHE Model

## Algorithm Technical Documentation

*A Compartmental Model of Infectious Disease Transmission*

**Language:** Python
**Algorithm:** SIDARTHE
**Date:** March 2026

## 1. Overview

This document describes the algorithmic implementation of the SIDARTHE calibration and prediction pipeline. The script reads observed compartmental data from training and test CSV files, fits the SIDARTHE eight-compartment ODE system to the training observations using bounded nonlinear least squares, and extends the fitted trajectory over the combined horizon for prediction and evaluation.

The ODE system is integrated numerically using a fourth-order Runge-Kutta (RK4) scheme. All model parameters are treated as time-invariant constants, distinguishing this implementation from the TV-SIDARHE variant in which transmission and detection rates are modulated by time-dependent factors.

| Module | Role |
|---|---|
| Data ingestion | Load training and test observations from CSV files; construct observation matrices |
| ODE system (prototype) | Define SIDARTHE continuous-time dynamics with eight state variables and 15 calibrated parameters |
| RK4 integrator (rks4) | Numerically solve the ODE system over a specified time horizon |
| Residual functions (rfun, rfun1) | Compute weighted fit error between model trajectory and observed data |
| Parameter optimisation | Minimise residuals via bounded least squares to recover calibrated parameters |

| Module | Role |
|---|---|
| Prediction and evaluation | Extend trajectory to full horizon; compute $R^2$ and MAPE; visualise fit and forecast |

# 2. Data Ingestion

## 2.1 Input Files

The pipeline requires two CSV files: `traindata.csv` and `testdata.csv`. Each must contain columns `D`, `R`, `H`, and `E`, corresponding to the four directly observable compartments. The Susceptible (S), Infected (I), Ailing (A), and Threatened (Th) compartments are treated as latent states and are not present in the input files.

**Note:** A commented-out line `# Th = np.array(data.Th)` appears in both data-loading blocks, indicating that the Threatened compartment was previously loaded as an observable but is excluded in the current implementation. It remains latent throughout the pipeline.

| Variable | Compartment | Status in Calibration |
|---|---|---|
| `D` | Diagnosed | Observable; loaded but excluded from loss (weight = 0 in rfun) |
| `R` | Recognised | Observable; sole calibration target (weight = 1 in rfun) |
| `H` | Healed | Observable; loaded but excluded from loss (weight = 0 in rfun) |
| `E` | Extinct | Observable; loaded but excluded from loss (weight = 0 in rfun) |

## 2.2 Array Construction

Each compartment column is extracted as a NumPy array and stacked into observation matrices:

```
y   = np.transpose(np.array([D, R, H, E]))     # shape: (n_train, 4)
y_t = np.transpose(np.array([D_t, R_t, H_t, E_t]))  # shape: (n_test, 4)
pre_long = y.shape[0] + y_t.shape[0]           # total simulation horizon
```

The combined length `pre_long` defines the total number of time steps for the post-calibration forward pass, spanning both training and test periods in a single continuous integration.

# 3. ODE System: prototype Function

## 3.1 Function Signature

The function `prototype(t, y, arg, N)` defines the right-hand side of the SIDARTHE ODE system

in continuous time. It takes the current time `t`, the state vector `y` (length 8), the parameter vector `arg` (length 15), and the total population `N`. It returns the derivative vector `[dS, dI, dA, dD, dR, dH, dTh, dE]`.

### 3.2 State Vector Layout

| Index | Variable | Compartment | Observable |
|---|---|---|---|
| y[0] | `S` | Susceptible | No (latent) |
| y[1] | `I` | Infected (undetected) | No (latent) |
| y[2] | `A` | Ailing (undetected) | No (latent) |
| y[3] | `D` | Diagnosed | Yes |
| y[4] | `R` | Recognised | Yes |
| y[5] | `H` | Healed | Yes |
| y[6] | `Th` | Threatened | No (latent in current pipeline) |
| y[7] | `E` | Extinct | Yes |

### 3.3 Parameter Vector Layout

The parameter vector `arg` has 15 elements. The mortality parameter $\tau$ (governing the Th → E flow) is fixed at $10^{-10}$ inside the function body and is not included in the optimised argument vector.

| Index | Symbol | Transition | Interpretation | Bounds |
|---|---|---|---|---|
| arg[0] | $\alpha$ | I → S (force) | Transmission rate from Infected (I) | [0, 1] |
| arg[1] | $\beta$ | D → S (force) | Transmission rate from Diagnosed (D) | [0, 1] |
| arg[2] | $\gamma$ | A → S (force) | Transmission rate from Ailing (A) | [0, 1] |
| arg[3] | $\delta$ | R → S (force) | Transmission rate from Recognised (R) | [0, 1] |
| arg[4] | $\varepsilon$ | I → D | Diagnosis rate of undetected infected | [0, 1] |
| arg[5] | $\zeta$ | I → A | Symptom onset rate (undetected) | [0, 1] |
| arg[6] | $\eta$ | D → R | Clinical progression: Diagnosed → Recognised | [0, 1] |
| arg[7] | $\theta$ | A → R | Clinical recognition of ailing cases | [0, 1] |
| arg[8] | $\mu$ | A → Th | Severe progression: Ailing → Threatened | [0, 1] |
| arg[9] | $\nu$ | R → Th | Severe progression: Recognised → | [0, 1] |

| Index | Symbol | Transition | Interpretation | Bounds |
|---|---|---|---|---|
| | | | Threatened | |
| arg[10] | $\lambda$ | I → H | Natural recovery rate from Infected | [0, 1] |
| arg[11] | $\kappa$ | A → H | Recovery rate from Ailing | [0, 1] |
| arg[12] | $\rho$ | D → H | Recovery rate from Diagnosed | [0, 1] |
| arg[13] | $\xi$ | R → H | Recovery rate from Recognised | [0, 1] |
| arg[14] | $\sigma$ | Th → H | Recovery rate from Threatened | [0, 1] |
| — (fixed) | $\tau = 10^{-10}$ | Th → E | Mortality rate from Threatened; fixed near zero | — |

### 3.4 Force of Infection

Four infectious compartments contribute to the depletion of the susceptible pool. The total force of infection at time t is:

$$\Lambda(t) = \alpha \cdot I(t) + \gamma \cdot A(t) + \beta \cdot D(t) + \delta \cdot R(t)$$

All four transmission rates are time-invariant constants, distinguishing this formulation from the TV-SIDARHE model where two of the four sources (I and A) are modulated by the time-decaying factor $b^{-I}$. The transmission contributions from D and R reflect partial isolation compliance.

### 3.5 Full ODE Equations

```
dS  = -(α·I + γ·A + β·D + δ·R) · S / N
dI  =  (α·I + γ·A + β·D + δ·R) · S / N  - (ε + ζ + λ)·I
dA  =  ζ·I - (θ + μ + κ)·A
dD  =  ε·I - (η + ρ)·D
dR  =  η·D + θ·A - (ν + ξ)·R
dH  =  λ·I + κ·A + ρ·D + ξ·R + σ·Th
dTh =  μ·A + ν·R - (σ + τ)·Th
dE  =  τ·Th
```

The population normalisation factor `N` appears in the `dS` and `dI` equations only, implementing a standard frequency-dependent (proportionate mixing) transmission formulation. All other flows are density-independent linear transitions between compartments. The conservation identity $S + I + A + D + R + H + Th + E = N$ is satisfied by construction at all times.

## 4. Numerical Integrator: rks4

### 4.1 Algorithm

The function `rks4(f, a, b, Za, M)` implements the classical fourth-order Runge-Kutta method over the interval [a, b] with M uniform steps. It returns the time grid `t` of shape (M+1, 1) and the state trajectory matrix `Z` of shape (M+1, 8).

| Argument | Type | Description |
|---|---|---|
| `f` | Callable | ODE right-hand side: f(t, y) → dy/dt, shape (8,) |
| `a` | float | Start of integration interval (set to 1 in all calls) |
| `b` | float/int | End of integration interval |
| `Za` | ndarray | Initial state vector at t = a, shape (8,) |
| `M` | int | Number of integration steps; step size h = (b − a) / M |

```
h = (b - a) / M
for i in range(1, M+1):
    k1 = h · f(t[i-1],          Z[i-1,:])
    k2 = h · f(t[i-1] + h/2,    Z[i-1,:] + k1/2)
    k3 = h · f(t[i-1] + h/2,    Z[i-1,:] + k2/2)
    k4 = h · f(t[i-1] + h,      Z[i-1,:] + k3)
    Z[i,:] = Z[i-1,:] + (k1 + 2·k2 + 2·k3 + k4) / 6
```

For all calibration calls the integration interval is [1, n_train] with M = n_train − 1, yielding a step size of h = 1. For the full prediction pass the interval is [1, pre_long] with M = pre_long − 1, maintaining the same unit step size. The `rks4` implementation is identical to that used in the TV-SIDARHE pipeline; the only difference is that the ODE right-hand side function is `prototype` rather than `SIDARHE`.

## 5. Initial Conditions

The eight-element initial state vector is constructed from the first observed data point and three fixed seed values for the latent compartments:

```
S  = N - y[0][0] - y[0][1] - y[0][2] - y[0][3] - 20000 - 10000 - 100
y0 = np.array([S, 20000, 10000, y[0][0], y[0][1], y[0][2], 100, y[0][3]])
#               S0    I0      A0      D0          R0          H0      Th0   E0
```

| Compartment | Initial Value | Source |
|---|---|---|
| $I_0$ (Infected) | 20,000 | Fixed seed value |
| $A_0$ (Ailing) | 10,000 | Fixed seed value |
| $Th_0$ (Threatened) | 100 | Fixed seed value |
| $D_0$ (Diagnosed) | `y[0][0]` | First row of training data |
| $R_0$ (Recognised) | `y[0][1]` | First row of training data |

| Compartment | Initial Value | Source |
|---|---|---|
| $H_0$ (Healed) | `y[0][2]` | First row of training data |
| $E_0$ (Extinct) | `y[0][3]` | First row of training data |
| $S_0$ (Susceptible) | $N - D_0 - R_0 - H_0 - E_0 - 20000 - 10000 - 100$ | Conservation identity |

Compared to the TV-SIDARHE initial conditions, the SIDARTHE pipeline adds a seed for the Threatened compartment ($Th_0 = 100$), which accounts for the eighth state variable. $S_0$ is derived by subtracting all seven other initial values from N = 140,005 × $10^4$, preserving the conservation identity $S + I + A + D + R + H + Th + E = N$ at t = 0.

# 6. Residual and Loss Functions

## 6.1 rfun — Vectorised Residual (used by least_squares)

The function `rfun(args, ydata, N)` integrates the ODE over the training interval and returns a residual vector of length n_train. Each element is the weighted absolute error between the model trajectory and the observed data at that step.

```
def rfun(args, ydata, N):
    n = ydata.shape[0]
    _, yval = rks4(lambda t,y: prototype(t,y,args,N), 1, n, y0, n-1)
    weights = np.array([0, 1])          # fit R only (index 1 within D,R slice)
    return np.sum(np.abs(yval[:,3:5] - ydata[:,0:2]) * weights, axis=1)
```

The slice `yval[:,3:5]` extracts columns D and R from the full 8-column state matrix. The corresponding observation slice is `ydata[:,0:2]` (columns D and R of the input matrix). The weight vector `[0, 1]` sets the D residual to zero and retains only the R residual, concentrating the optimisation signal on the Recognised compartment. The returned vector of shape (n_train,) is passed to `scipy.optimize.least_squares`, which minimises its sum-of-squares norm.

## 6.2 rfun1 — Scalar Loss (auxiliary)

The auxiliary function `rfun1(arg)` unpacks all 17 positional elements from its argument (including two unused placeholders beyond the 15 fitted parameters) and computes a single scalar weighted loss via matrix dot product:

```
weights = np.array([[1],[1],[0],[0],[0]])
return np.sum(np.dot(np.abs(yval[:,3:] - y), weights))
```

This function uses `yval[:,3:]`, the five observable columns (D, R, H, Th, E) from the 8-column state matrix, compared against all columns of the training observation matrix `y`. The weight vector assigns unit weight to D and R, and zero weight to the remaining three columns. `rfun1` is defined but not

called by the main optimisation loop in the current implementation; it is available for manual diagnostics or scalar-loss-based search.

**Note:** There is a dimensional asymmetry between `rfun` and `rfun1`. In `rfun` the state slice is `yval[:,3:5]` (columns D, R only), whereas in `rfun1` it is `yval[:,3:]` (all five observable columns D, R, H, Th, E). The two functions are therefore not interchangeable; `rfun` is the sole function used for parameter estimation.

# 7. Parameter Optimisation

## 7.1 Solver Configuration

Parameters are estimated by calling `scipy.optimize.least_squares` with the vectorised residual function `rfun`. The solver uses a trust-region reflective algorithm with finite-difference Jacobian approximation.

```
arg0 = np.array([0.1779, 0.0048, 0.0048, 0.3926, 0.1238, 0.0430, 0.0430,
                 0.1799, 0.0499, 0.1480, 0.1480, 0.0499, 0.1480, 0.0255, 0.0336])

optiargs = least_squares(
    lambda arg: rfun(arg, y, N),
    arg0,
    bounds = ((0,)*15, (1,)*15),
    max_nfev = 10000
)
```

| Setting | Value | Notes |
|---|---|---|
| Parameter count | 15 | All rates; τ is fixed and excluded |
| Initial parameters (arg0) | See code | Set to reference SIDARTHE values from Giordano et al. (2020) |
| Lower bounds | 0 for all 15 parameters | Prevents negative transition rates |
| Upper bounds | 1 for all 15 parameters | All rates are bounded as per-unit-time probabilities |
| Max function evaluations | 10,000 | Termination criterion; increase if convergence is not achieved |
| Method | Trust Region Reflective (default) | Handles box constraints; Jacobian estimated by finite differences |

All 15 parameters share the same upper bound of 1, unlike the TV-SIDARHE pipeline where the bases a and b are unbounded above. The initial values correspond to the published reference parameterisation of the SIDARTHE model.

### 7.2 Optimised Parameters

On convergence, the calibrated parameter vector is stored in `optiargs.x`. The full result object `optiargs` is printed to console and contains the final residual vector, cost, Jacobian, and convergence status. The optimised parameters are then used in a single forward RK4 integration over the full `pre_long` horizon.

## 8. Prediction and Evaluation

### 8.1 Full-Horizon Forward Pass

After calibration, the ODE is integrated once over the combined training and test horizon:

```
tval, yval = rks4(
    lambda t, y: prototype(t, y, optiargs.x, N),
    1, pre_long, y0, pre_long - 1
)
```

The resulting matrix `yval` has shape (pre_long, 8). The training portion corresponds to rows `0 : n_train` and the prediction portion to rows `n_train : pre_long`. The Recognised compartment trajectory is extracted as `yval[:,4]` (column index 4 in the 8-column state matrix).

### 8.2 Evaluation Metrics

| Metric | Formula | Applied to |
|---|---|---|
| $R^2$ (coefficient of determination) | `r2_score(yval[:n,4], y[:,1])` | Training period, R compartment |
| MAPE (mean absolute percentage error) | `mean(\|y_pred - y_true\| / y_true)` | Test period, R compartment |

Both metrics target column index 4 of `yval` (Recognised, R) matched against column index 1 of the observation arrays `y` and `y_t` respectively. Both values are printed to console with three significant figures. Note that $R^2$ is computed from `r2_score` but is referenced through the variable `R2`, which is assigned inside a commented block earlier in the script; the `print` statement at the end will raise a `NameError` if that line remains commented out during execution.

### 8.3 Visualisation

The script renders a single Matplotlib figure overlaying three curves on the same axes:

| Curve | Line Style | Data Source | Time Range |
|---|---|---|---|
| Fit | Red dashed | `yval[:,4]` | Full horizon (training + test) |

| Curve | Line Style | Data Source | Time Range |
|---|---|---|---|
| True (training) | Black dashed | `y[:,1]` | Training period only |
| Test (held-out) | Yellow solid | `y_t[:,1]` | Test period only |

The x-axis label is `'days'`. All curves represent the Recognised (R) compartment. The legend is placed at the best available position. Seaborn's default theme is applied via `sns.set()` before rendering.

# 9. Dependencies and Execution

## 9.1 Required Packages

| Package | Usage |
|---|---|
| numpy | Array operations, ODE state matrix |
| pandas | CSV ingestion |
| scipy.optimize.least_squares | Bounded nonlinear parameter calibration |
| sklearn.metrics.r2_score | Training-period goodness-of-fit |
| matplotlib / seaborn | Trajectory visualisation |
| math | Not used directly in prototype; available in scope |
| time | Wall-clock timing of the full pipeline |

## 9.2 Execution

Place `traindata.csv` and `testdata.csv` in the working directory and execute:

```
python sidarthe_calibration.py
```

# References

**[1]** Giordano, G., Blanchini, F., Bruno, R., Colaneri, P., Di Filippo, A., Di Matteo, A., & Colaneri, M. (2020). Modelling the COVID-19 epidemic and implementation of population-wide interventions in Italy. *Nature Medicine*, 26(6), 855–860.\

**[2]** Virtanen, P., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.

**[3]** Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

**[4]** Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.